

Lunar Lander

Ahmadreza Tavasoli Bahabadi

Introduction and Project Objective

The objective of this project is to implement and train an intelligent agent to control and successfully land a spacecraft in the **LunarLander** environment. This problem is considered one of the classic Reinforcement Learning problems, in which the agent must learn, by receiving feedback from the environment, to maximize the received reward while maintaining balance and using fuel efficiently, and to land precisely between the two designated flags. To implement this system, an enhanced version of the **Deep Q-Network (DQN)** algorithm, along with advanced stabilization mechanisms, has been used.

1 Project Structure

The implemented project is designed based on a modular and separated structure to facilitate code readability, maintenance, and further development:

- **game.py**: The interface module for the game environment and management of communication with the **API** of the **Gymnasium** environment.
- **model.py**: Defines the architecture of the deep neural network (**PyTorch**).
- **agent.py**: Implements the logic of the intelligent agent, including the policy network, target network, replay memory, and soft update algorithm.
- **train.py**: Centrally manages hyperparameters, the main training loop, weight updates, and early stopping evaluation.
- **test.py**: Executes the trained model for final evaluation under randomized conditions.

2 Comparison of Input with Classical DQN Models

In classical DQN implementations (such as the 2013 paper and the **Nature** 2015 paper on Atari games), the model input consisted of image frames with dimensions of $N \times M$ pixels. In those architectures, to extract the state and remove redundant dimensions, the image was first passed through a feature transformation function (such as ϕ), and then four consecutive frames were stacked together to capture concepts such as velocity and direction of movement.

In the **LunarLander** environment, image processing is not necessary, since at each time step, the environment directly provides an **8-component state vector**:

$$s = [x, y, v_x, v_y, \theta, \omega, c_1, c_2]^T \quad (1)$$

This vector fully contains the spatial coordinates (x, y) , the linear velocity components (v_x, v_y) , the spacecraft’s angle and angular velocity (θ, ω) , and two binary variables indicating whether the spacecraft’s left and right legs (c_1, c_2) are in contact with the lunar surface.

3 Deep Q-Network Neural Network Architecture

The neural network that maps the state to the values $Q(s, a)$ is a deep neural network with the following specifications:

- **Input layer:** Contains 8 neurons corresponding to the dimensionality of the environment state vector.
- **Hidden layers:** Two consecutive fully connected hidden layers, each containing 128 neurons, together with nonlinear **ReLU** activation functions and weight initialization using the **Kaiming Normal** method.
- **Output layer:** Contains 4 neurons corresponding to the Q -values for each of the available actions, with **Xavier Uniform** initialization.

In choosing this architecture, 128 neurons for the hidden layers provide an optimal balance; too many neurons lead to overfitting, while too few make the model overly simple and prevent it from fully learning the complexities of the environment. In addition, the use of the **ReLU** activation function, due to its fast computation and its ability to mitigate the vanishing gradient problem, improves the stability and convergence speed of the network.

4 Training Stabilization Mechanisms in the DQN Algorithm

4.1 Experience Replay Buffer

In traditional online **Q-Learning** algorithms, consecutive samples have strong temporal correlation, which can lead to divergence, gradient explosion, and becoming trapped in positive feedback loops. To address this challenge, a replay buffer is used to store 5-tuples of experience:

$$e_t = (s_t, a_t, r_t, s_{t+1}, \text{done}) \quad (2)$$

At each training step, random samples are drawn from this memory, after which the backpropagation process is performed to break the correlation between samples.

4.2 Exploration and Exploitation Strategy (ϵ -Greedy Policy)

To establish a balance between exploration and exploitation, the ϵ -Greedy strategy is used. With probability ϵ , the agent selects a random action, and with probability $1 - \epsilon$, it selects the best action predicted by the network. The value of ϵ is decreased at the end of each episode according to the following exponential formula:

$$\epsilon_{t+1} = \max(\epsilon_{\min}, \epsilon_t \times \text{decay}) \quad (3)$$

The reason for using the max function and comparing it with $\epsilon_{\min}$ is to ensure that a minimum percentage of exploration ($\epsilon_{\min} = 0.01$) always remains in the model, so that the agent does not completely freeze its knowledge.

4.3 Solving the Moving Target Problem with Soft Updates (Soft Target Update)

In classical DQN implementations (such as the `Nature` 2015 paper), the target network weights (θ^-) are kept fixed and are replaced by the weights of the main network every C steps using a hard update ($\theta^- \leftarrow \theta$). This one-time copying at specified intervals causes sudden changes in the Bellman target values and consequently creates severe fluctuations and shocks in the loss function.

In this project, the **Soft Target Update** method based on **Polyak Averaging** is used. In this mechanism, at every time step, the target network weights are continuously, smoothly, and gradually combined with the main network weights using a specified coefficient, preventing the shocks caused by periodic updates:

$$\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^- \quad (4)$$

where $\tau \ll 1$ ($\tau = 0.001$) is the soft update coefficient. This change transforms the Bellman target into a “smooth moving target,” eliminates the shocks caused by periodic copying, and, while significantly reducing fluctuations in the loss function, improves the stability and convergence speed of the model.

5 Complete Training and Evaluation Process of the Algorithm

The training process of the intelligent agent in the `LunarLander` environment is performed through an iterative loop based on the following steps:

5.1 1. Initialization

Before the training loop begins, the following items are loaded into the program memory:

- **Neural networks:** Creation of the policy network with initial weights θ and the target network with weights θ^- . Initially, the target network weights are set exactly equal to those of the main network ($\theta^- \leftarrow \theta$).
- **Replay Buffer:** Creation of a circular queue with a capacity of $N = 100,000$.
- **Hyperparameters:** Setting the learning rate $\alpha = 3 \times 10^{-4}$, discount factor $\gamma = 0.99$, τ rate $= 10^{-3}$, and early stopping threshold of 240.0.

5.2 2. Main Training Loop and Step Truncation

In each episode $e = 1, 2, \dots, E$, the following steps are performed:

1. **Environment reset:** The environment is initialized and the initial state s_0 is received.

2. **Time-step loop:** At each step t , the action a_t is selected according to the ϵ -Greedy policy, executed in the environment, and the outputs $(s_{t+1}, r_t, \text{done})$ are received.
3. **Step Truncation:** To prevent infinite hovering behavior and wasted training time, the maximum number of time steps per episode is limited to 500 steps. If the number of steps exceeds this limit, the episode is terminated and the **done** signal is activated.

5.3 3. Batch Extraction and Continuous Weight Updates

If the data in the memory has reached the batch size (**Batch Size** = 128), the following actions are performed at each time step:

1. **Random sampling:** A batch of B experiences is sampled.
2. **Calculation of the Bellman target (y_i):**

$$y_i = r_i + \gamma \cdot (1 - d_i) \cdot \max_{a'} Q_{\theta^-}(s'_i, a') \quad (5)$$

3. **Loss calculation and backpropagation:** The mean squared error (MSE) is calculated and the weights θ are updated together with gradient clipping using a threshold of 10.0 to prevent gradient explosion:

$$L(\theta) = \frac{1}{B} \sum_{i=1}^B (y_i - Q_{\theta}(s_i, a_i))^2 \quad (6)$$

4. **Soft target network update:** The equation $\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$ is immediately applied at the end of each training step.

5.4 4. Moving Average Early Stopping

At the end of each episode, ϵ is decreased and the moving average of the rewards over the most recent 100 episodes is calculated:

$$\bar{R}_{100} = \frac{1}{100} \sum_{k=e-99}^e R_k \quad (7)$$

If $\bar{R}_{100} \geq 240.0$, the training process is stopped. Choosing a threshold of 240.0 (instead of the standard threshold of 200.0) ensures that the agent achieves not only a collision-free landing, but also **optimal fuel consumption and landing speed**. Additionally, setting this precise threshold prevents the occurrence of overfitting.

5.5 5. Final Evaluation under Randomized Conditions (Randomized Evaluation)

During the evaluation stage (**test.py**), the saved weights are loaded and $\epsilon = 0$ is set. To realistically evaluate the agent's generalization ability, the environment's **reset()** function is executed without a fixed initialization (Fixed Seed), so that the agent is evaluated under completely random and previously unseen spatial and velocity conditions.

6 Project Optimized Hyperparameters Table

All key hyperparameters of the algorithm are configured in a centralized dictionary in the `train.py` file, and their optimized values are summarized in Table 1.

Table 1: Hyperparameters tuned for the DQN algorithm

Hyperparameter Name	Performance Description	Optimized Value
Total Episodes	Maximum number of training episodes	2500
Target Moving Avg	Moving average threshold for early stopping	240.0
Max Steps per Episode	Maximum allowed steps per episode (Truncation)	500
Hidden Dimension	Number of neurons in each hidden layer of the network	128
Learning Rate (α)	Learning rate of the Adam optimizer	3×10^{-4}
Discount Factor (γ)	Discount factor for future rewards	0.99
Soft Update Rate (τ)	Soft update coefficient of the target network	0.001
Batch Size	Size of batches sampled from memory	128
Buffer Capacity	Maximum capacity of the experience replay memory	100,000
Epsilon Decay	Exponential decay factor of the exploration rate per episode	0.995
Epsilon Min	Lower bound of the exploration rate ($\epsilon_{\min}$)	0.01

7 Algorithm Training Progress Plot

The plot in Figure 1 shows the changes in the scores of each episode and the moving average over the most recent 100 episodes throughout the training process. As shown in the plot, the moving average smoothly and without severe fluctuations reaches the threshold of 240.0.

Reason for Using Experience Replay

There are several reasons for using experience replay. In the DQN algorithm, at each update step and during the use of the backpropagation algorithm, a number of data samples are used. Using experience replay allows us to use the same data multiple times and make maximum use of the information obtained.

The second reason is that, in algorithms for updating the parameters of a neural network, it is assumed that the data are independent of one another. If we use only the most recently obtained data and, for example, always take the last 100 data points, there is a high probability of becoming trapped in a positive feedback loop, which ultimately leads to getting stuck in local optima. For example, the algorithm may find an average-quality action at one stage, causing it to affect the policy at the next stage, since that exact action was used in updating the network. Subsequently, this policy is reinforced, and each time the probability of selecting that average-quality action increases. However, there may be another much better action several steps later that could be discovered through a different

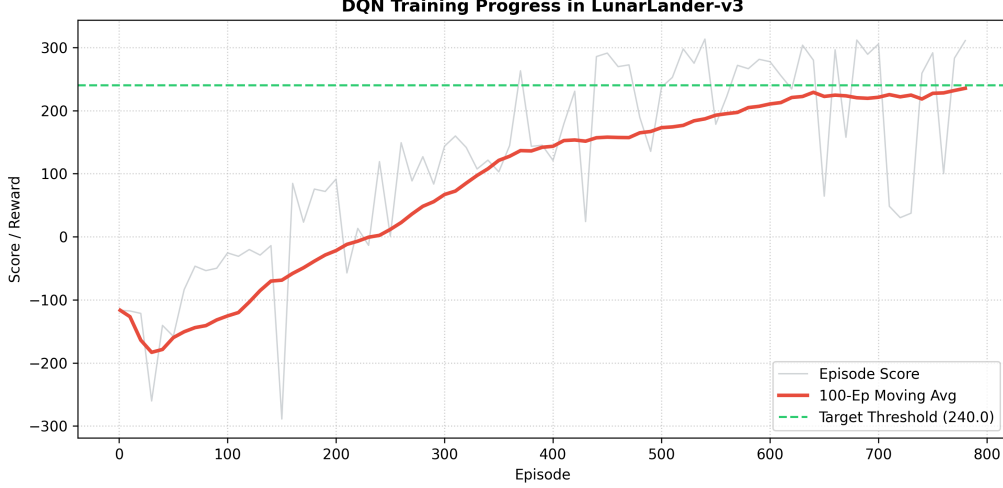


Figure 1: Learning curve of the DQN algorithm and the growth of the moving average of rewards

trajectory, but this positive feedback loop somehow causes the algorithm to become blind to it.

Reason for Using a Fixed Target and the Moving Target Challenge

In calculating the target value (y_i), we use the following relation:

$$y_i = r_i + \gamma \max_{a'} Q(s'_i, a'; \theta) \quad (8)$$

During the calculation of the gradient of the loss function, we seek to update the parameters so that the predicted value $Q(s_i, a_i)$ becomes closer to the target value (y_i). However, the main challenge is that y_i itself is not constant and contains the function Q within its formula. Therefore, every time the value of Q changes and is updated, the target value (y_i) also shifts. It is exactly as if we were moving toward a target that itself moves every time we take a step toward it.

Of course, not keeping y_i fixed does not mean that the problem is absolutely unsolvable, but it greatly increases the likelihood of divergence or a significant increase in the number of steps required for convergence. Keeping the target function fixed (or, as in the implemented algorithm, going one step further and updating the target function softly at a slower rate) prevents this chase from occurring and makes the training process completely stable.

Reason for Using Epsilon and the Difference Between Different Epsilon Values

The epsilon variable is introduced into the DQN algorithm so that we can simultaneously search through the state space while making use of the experiences and policy learned so far. If epsilon is very small or decreases too rapidly, the model will have a strong tendency toward exploitation and toward using the policy it has learned so far. This may bias the model because the initial policy is very likely not optimal, and when the model

relies heavily on the learned policy, the data repeatedly stored in the memory will also be generated according to this same policy. As a result, the policy may become reinforced over time, even though it is not necessarily optimal.

On the other hand, if epsilon decreases too slowly, the model’s tendency to act randomly will remain high and the training time will increase exponentially. The model will fail to learn from its mistakes and may repeatedly visit states that are already lost, causing episodes and data to be wasted.

Analysis of Q -Value and Network Behavior During a Crash

The output of the last layer of the neural network is a 4-dimensional vector of Q -values for the current input state s . Each output neuron represents the expected value of future discounted rewards conditioned on selecting the corresponding action and following the agent’s policy in subsequent steps:

$$Q(s, a) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \mid s_t = s, a_t = a \right] \quad (9)$$

If the spacecraft is at a high altitude and falling freely at high speed, it is reasonable to expect the Q -value for the “activate main engine” action (action 2) to be significantly greater than those of the other actions ($Q(s, a_2) \gg Q(s, a_{\text{others}})$).

During training, the model learns that crashing results in a negative score (penalty). This causes changes in the Q values of states near the ground as the spacecraft moves downward. In subsequent training episodes, this negative reward propagates among neighboring states, and in this way the model gradually estimates the Q -values of states in which the spacecraft is in free fall to be very high for the “activate main engine” action.